Universidad Central de Venezuela

Facultad de Ciencias

Escuela de Computación

Organización y Estructura del Computador I

Diseño y Simulación de Procesador RISC-16 (OECI 2-2025) – Primera Etapa

Introducción

El objetivo de esta primera etapa del proyecto es que a partir de las especificaciones de una arquitectura del conjunto de instrucciones (ISA) de un procesador de 16 bits, se debe desarrollar su ensamblador (software), el cual debe ser capaz convertir de lenguaje ensamblador a código máquina binario.

En esta entrega se integra conceptos de diseño lógico, gestión de interrupciones y ensamblador.

En las etapas dos y tres se abordaran otros conceptos

Especificación de la Arquitectura (ISA)

Ancho de Palabra: 16 bits para datos e instrucciones.

Espacio de Direcciones: 16 bits.

Banco de Registros: 8 Registros de Propósito General (R0 − R7) de 16 bits cada uno.

- **R0 (Zero):** Registro cableado a cero. Siempre lee 0; la escritura es ignorada.
- **R1 al R6:** Registros de proposito general.
- **R7 (SP):** Puntero de Pila (Stack Pointer) por convenio.

Registro de Estado (Program Status Register - PSR)

A diferencia de arquitecturas simples, este procesador incluye un registro de banderas de 4 bits (no accesible directamente como R0 − R7) que se actualiza tras operaciones aritméticas y lógicas para permitir saltos condicionales complejos.

Banderas:

Z	N	C	V

1. **Z** (Zero): 1 si el resultado es cero.

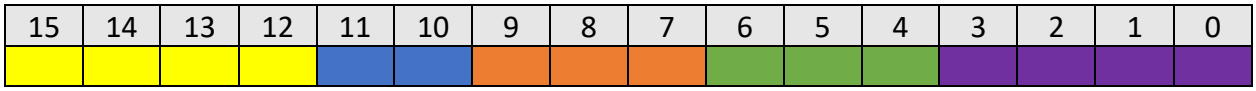
2. **N** (Negative): 1 si el resultado es negativo.

3. **C** (Carry): 1 si hay acarreo de salida (aritmética sin signo).

4. **V** (Overflow): 1 si hay desbordamiento en complemento a 2.

Formato de Instrucciones

Todas las instrucciones tienen una longitud fija de 16 bits. Se definen tres formatos base:



Campo	Bits	Descripción
COP	15 – 12	Código de operaciones
Tipo	11 – 10	Formato de instrucción R (00), I (01), J (10)
Rs1	9 – 7	Registro fuente
Rs2/Inm	6 – 4	Registro fuente o parte alta del inmediato
Rd/Inm	3 – 0	Registro destino o parte baja del inmediato

Conjunto de Instrucciones (Instruction Set)

Tipo R (Registro-Registro)

Formato: COP (4) | 00 (2) | Rs1 (3) | Rs2 (3) | Rd (4) (**Nota:** Rd ocupa 3 bits, queda un bit restante reservado o para extensiones).

#	Nemónico	Operación	Descripción	Actualiza PSR
0	ADD	Rd = Rs1 + Rs2	Suma Entera	Si
1	SUB	Rd = Rs1 – Rs2	Resta Entera	Si
2	AND	Rd = Rs1 & Rs2	AND Lógico bit a bit	Z, N
3	ORR	Rd = Rs1‖ Rs2	OR Lógico bit a bit	
4	CMP	Rs1 – Rs2	Compara (Resta sin guardar resultado)	Si
5	LSL	Rd = Rs1 << Imm	Logical Shift Left (Desplazamiento)	Z, N, C
6	LSR	Rd = Rs1 >> Imm	Logical Shift Right	Z, N, C

7	ASR	Rd = Rs1 >>> Imm	Arithmetic Shift Right (mantiene signo)	Z, N, C

Tipo I (Inmediato y Memoria)

Formato: COP (4) | 01 (2) | Rs1 (3) | Inmediato/Offset (7 bits)

#	Nemónico	Operación	Descripción
0	ADDI	Rd = Rs1 + Imm	Suma con inmediato (con extensión de signo)
1	SUBI	Rd = Rs1 - Imm	Resta con inmediato (con extensión de signo)
2	ANDI	Rd = Rs1 & Imm	AND Lógico bit a bit con immediato
3	ORI	Rd = Rs1‖ Rs2	OR Lógico bit a bit con immediato
4	LW	Rd = Mem[Rs1 +Off]	Carga palabra desde memoria (Load Word)
5	SW	Mem[Rs1 +Off] = Rd	Almacena palabra en memoria (Store Word)

Tipo J (Control de Flujo y Saltos)

Formato: COP (4) | 10 (2) | Condición/Reg (3) | Offset Relativo (7 bits)

Nota: Todos los saltos deben implementar direccionamiento Relativo al PC (Target = PC actual + Offset).

#	Nemónico	Condición	Descripción
0	JMP	-	Salto Incondicional Relativo
1	BEQ	Z = 1	Branch if Equal (Salta si igual)
2	BNE	Z = 0	Branch if Not Equal (Salta si distinto)
3	BGT	Z = 0 ∧ N = V	Branch if Greater Than (Con Signo)
4	JAL	-	Jump and Link: Guarda PC + 1 en R7, Salta a Target
5	RET	-	Retorno de subrutina: PC = R7
6	RETI	-	Retorno de Interrupción (Restaura Contexto)
7	HALT	-	Detiene el procesador

Características/Requerimientos del Entregable (Software)

1. El programa (software) se debe implementar en C++.
2. La entrada al programa es un código en lenguaje ensamblador el cual debe traducir a código de máquina. Para ello debe permitir entrada por teclado y archivo, debe aceptar archivos con extensión “.asm”, el resultado debe generar un archivo similar a un “.bin” pero solo texto con extensión “.txt”.
3. Se debe construir la tabla de simbolos (Etiquetas y Direcciones) y mostrarla en el comienzo del archivo “.txt”.

4. Generar el código objeto (binario) y realizar la resolución de direcciones.
5. El ensamblador desarrollado debe calcular automáticamente el Offset necesario, donde Offset = (Dirección de Etiqueta) – PC.
6. Se requiere proveer soporte para directivas .org, .word (reservar 16 bits), .space (reservar bloque) y .string.
7. La dirección de inicio del código de máquina son los 16bits menos siginificativos del resultado de la suma de las cédulas de los integrantes del equipo (colocar el cálculo y procedimiento).
8. Debe soportar nemónicos que no existen en hardware
    a. MOV Rd, Rs -> Traducir a ADD Rd, Rs, R0
    b. IMOV Rd,$1 (immediato) -> Traducir a ADD Rd, $1, R0
    c. NOP -> Traducir a ADD R0, R0, R0
9. En la parte final del archivo ".txt" debe mostrar el contenido final de los registros incluyendo el PSR
10. Mostrar el resultado de la traducción en tres columnas

Dirección | Código de máquina en hexadecimal | # instrucción ensamblador

Correo a enviar y adjuntar lo siguiente

1. Código fuente de su programa bien documentado, se penalizara fuertemente el **abuso** de **IA**, su código tiene que ser compilado con gcc en linux, el archivo identificarlo como "grupo-#-cod-f1.cc"
2. Archivo ejecutable en un entorno linux identificado como "grupo-#-exe-f1"
3. Un informe breve explicando como llegaron a su propuesta de software, incluir cuales fueron sus retos y como lo superaron, el informe identicarlo como "grupo-#-informe-f1.pdf"
4. El asunto del correo es "proyecto-grupo-#-fase-1"

Fechas

12/12/2025 Publicación de la primera etapa

20/01/2026 Entrega primera etapa y publicación de segunda etapa

03/02/2026 Entrega segunda etapa y publicación de tercera etapa

18/02/2026 Entrega final